

【实验报告】

课程名： 计算机组成与体系结构（H）

实验名称： lab6 异常和中断

学生： 杨箫羽

学号： 24300240069

时间： 2026 年 6 月 9 日

一、实验内容与实现思路说明

实验 6 要求增加完整的异常处理和中断。为了实现异常处理和中断，我在 lab5 的基础上复用 ecall 的处理流程，通过 trap_valid 对所有异常中断进行统一处理。对于异常，分别在 IF 阶段、ID 阶段和 MEM 阶段识别三种异常，在异常信号推进到 WB 阶段时进行统一处理。对于中断，在中断信号到来时进行处理。

本次实验在 instr_mem 模块和 data_mem 模块都进行了功能补充，因此对 lab1-5 维持的 cpu 各个模块进行了重构。把 instr_mem 模块调整为 4 级有限状态机。在 mmu 增加 flush，梳理 valid pc unit 各种 pc 跳转的优先级别。

下面分总体电路图、做出重要修改的模块和 bonus 解答进行说明。

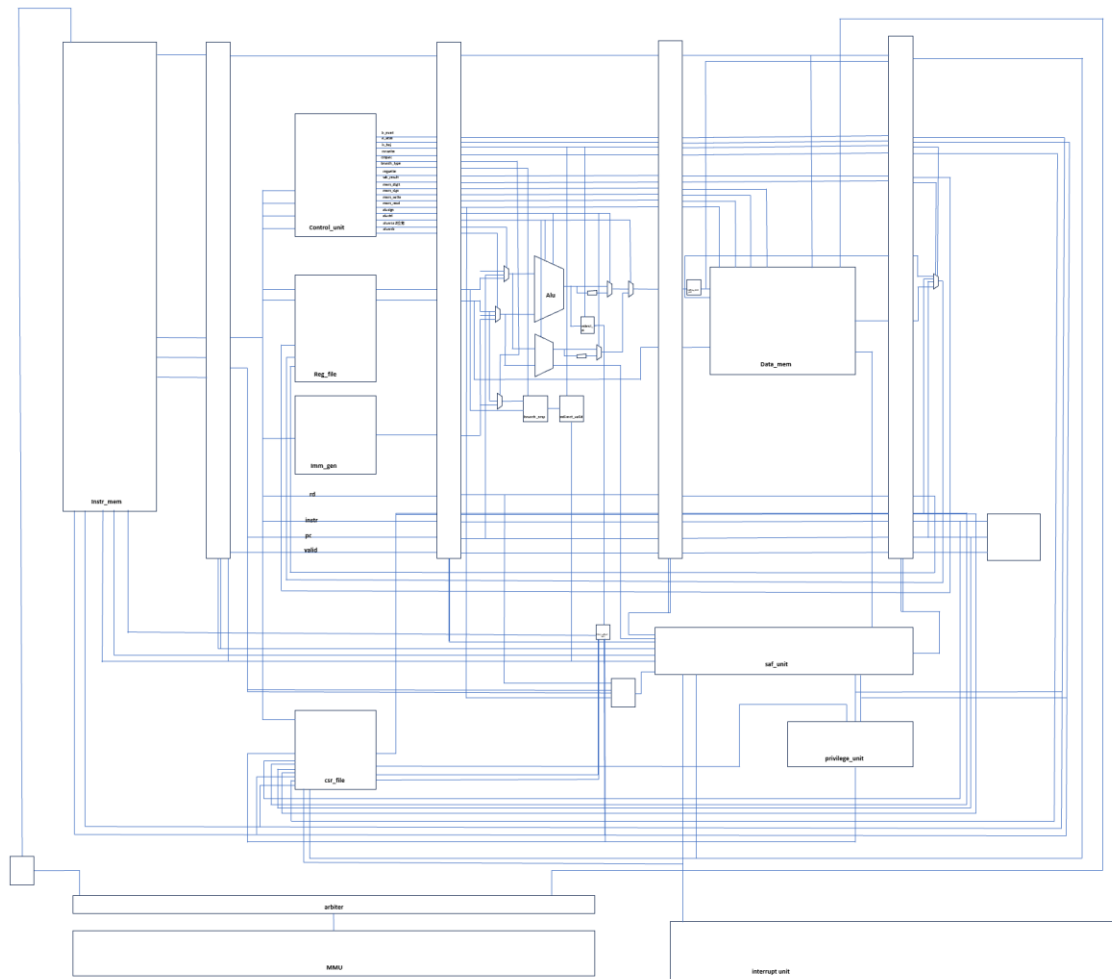
[illegible]

```

Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Timer interrupt in test_trap, this should happen 50 times.
Test m_trap [OK]
Privileged test finished.
Exit with code = 0

```

二、总体电路图



三、重要修改模块说明。

1、instr_mem 模块重新组织

接口：

```
module instr_mem import common::*;(
  input logic      clk,
  input logic      reset,

  input logic      consume,
  input logic [63:0] pcinit,

  input ibus_resp_t ibus_resp,
  output ibus_req_t  ibus_req,

  output logic      instr_valid,
  output logic [31:0] instr,
  output logic [63:0] pc,

  input logic [63:0] redirect_pc,
  input logic      branch_redirect_valid,
  input logic      is_ecall,
  input logic      is_mret,

  input logic      pc_stall,

  output logic      iaddr_exc
);
```

功能：实现取指前端的 4 级有限状态机（NER_INSTR-CHECKING-EXEC-COMMIT 四个阶段）。复位后从 pcinit 开始，状态依次为 **NEW_INSTR** 准备 PC、**CHECKING** 检查地址是否按照取指类型对齐、**EXEC** 向 ibus 发起取指请求、**COMMIT** 输出稳定的 pc/instr/iaddr_exc 并拉高 instr_valid。若地址不对齐则直接提交异常。遇到分支重定向时更新 pc_prepared 并重新检查；若 pc_stall 有效，状态和输出保持，同时锁存重定向，避免丢失跳转。consume 表示下游已接收当前指令，可进入下一条取指。

FSM:

```
always_comb begin
    next = cur;

    unique case (cur)
        NEW_INSTR: begin
            if (!pc_stall)
                next = CHECKING;
        end

        CHECKING: begin
            if (pc_stall)
                next = CHECKING;
            else if (effective_redirect_valid)
                next = NEW_INSTR;
            else if (pc_misaligned)
                next = COMMIT;
            else
                next = EXEC;
        end

        EXEC: begin
            if (pc_stall)
                next = EXEC;
            else if (effective_redirect_valid)
                next = NEW_INSTR;
            else if (fetch_ok)
                next = COMMIT;
        end

        COMMIT: begin
            if (pc_stall)
                next = COMMIT;
            else if (effective_redirect_valid)
                next = NEW_INSTR;
            else if (consume)
                next = NEW_INSTR;
        end

        default: begin
            next = NEW_INSTR;
        end
    endcase
end
```

2、saf_unit

接口:

功能: saf_unit 是流水线冒险处理与控制同步模块, 负责统一生成各级寄存器的 stall 和 flush 信号。对于 mem_stall, 整个流水线暂停; 对于 load_use_stall, 暂停 PC 和 IF/ID 级, 同时通过清空 ID/EX 插入 Bubble 解决数据相关。发生分支跳转、异常或 mret 时, 通过 control_redirect 刷新前端错误路径指令, 其中分支仅清除

```
module saf_unit (
    input  logic mem_stall,
    input  logic load_use_stall,

    input  logic branch_redirect_valid,
    input  logic trap_valid,
    input  logic mret_valid,

    output logic pc_stall,
    output logic if_id_stall,
    output logic id_ex_stall,
    output logic ex_mem_stall,
    output logic mem_wb_stall,

    output logic if_id_flush,
    output logic id_ex_flush,
    output logic ex_mem_flush,
    output logic mem_wb_flush
);
```

IF/ID、ID/EX，而 trap/mret 还会进一步清除 EX/MEM，保证所有更年轻指令失效。MEM/WB 不被刷新，避免正在提交的 trap/mret 指令被自身误清除，从而保证异常与返回控制流的正确提交。

3、datapath 修改部分

```
always_comb begin
    trap_cause = 64'b0;

    if (iaddr_exc_w) begin
        trap_cause = 64'd0; // instruction address misaligned
    end
    else if (instr_exc_w) begin
        trap_cause = 64'd2; // illegal instruction
    end
    else if (daddr_exc_w) begin
        if (instr_w[6:0] == 7'b0100011)
            trap_cause = 64'd6; // store address misaligned
        else
            trap_cause = 64'd4; // load address misaligned
        end
    end
    else if (valid_w && is_ecall_w) begin
        if (privl_mode == 2'b00)
            trap_cause = 64'd8; // ecall from U
        else
            trap_cause = 64'd11; // ecall from M
        end
    end
    else if (exint_take) begin
        trap_cause = 64'd11; // machine external interrupt
    end
    else if (trint_take) begin
        trap_cause = 64'd7; // machine timer interrupt
    end
    else if (swint_take) begin
        trap_cause = 64'd3; // machine software interrupt
    end
end
```

异常、中断与 branch 指令统一处理。首先检测取指地址异常，包括取指阶段产生的 iaddr_exc_e 和分支跳转目标地址未对齐产生的 redirect_iaddr_exc_e，统一传递到后续流水级处理。随后在 W 阶段判断同步异常（取指异常、非法指令、数据地址异常、ecall）和 mret 事件；同时根据 mstatus.MIE、mie、mip 以及外部/定时器/软件中断信号决定是否响应中断。为避免刚到达的中断被遗漏，使用 mip_next_for_int 预先合成新的 pending 状态。最终生成统一的 trap_valid、trap_cause 和 trap_pc，并按 RISC-V 优先级确定异常原因。

4、csr_file 修改部分

```
if (trap_valid) begin
    csr_mstatus[12:11] <= trap_priv;
    csr_mstatus[7] <= csr_mstatus[3];
    csr_mstatus[3] <= 1'b0;

    csr_mepc <= trap_pc;
    csr_mcause <= {trap_is_interrupt, trap_cause[62:0]};
    csr_mtval <= 64'b0;
end
else if (mret_valid) begin
    csr_mstatus[3] <= csr_mstatus[7];
    csr_mstatus[7] <= 1'b1;
    csr_mstatus[12:11] <= 2'b00;
    csr_mstatus[16:15] <= 2'b00;
end
```

csr_file 在异常和中断发生时负责更新机器级 CSR。每拍先递增 mcycle，并把 swint/trint/exint 同步到 mip[3/7/11]。当 trap_valid 有效时，优先进入 trap 处理：保存当前特权级到 mstatus.MPP，把全局中断使能 MIE 保存到 MPIE，再关闭 MIE，防止嵌套中断；同时将恢复地址写入 mepc，将异常/中断原因写入 mcause，其中最高位表示是否为中断，低位保存 trap_cause。mtval 此处置零。若执行 mret，则恢复 MIE=MPIE，重新置位 MPIE，并清空 MPP 等状态。CSR 普通写入优先级低于 trap/mret，避免异常现场被软件写覆盖。

四、问题解答

1、编写中断处理程序，在时钟中断时打印一些内容。

```
# set_timer: 设置下一次时钟中断
li      t0, 0x3800bff8
ld      t1, 0(t0)
addi    t1, t1, 20
li      t0, 0x38004000
sd      t1, 0(t0)

m_test_trap_entry:
    la    a0, msg
    call  printf

    call  set_timer

    mret

.section .rodata
msg:
    .asciz "Timer interrupt in test_trap\n"
```

在 `set_timer` 中通过 MMIO 地址读取当前 `mtime`，加 20 后得到下一次中断触发时间。将这个值写入地址 `0x38004000` 对应的 `mtimecmp`，表示当 `mtime` 增长到这里是产生中断。

中断发生时，CPU 跳转到 `m_test_trap_entry`。入口中，程序将 `msg` 字符串地址放入 `a0`。调用 `printf` 输出。打印完成后调用 `set_timer` 重新设置下一次时钟中断，最后执行 `mret` 返回被中断的程序继续执行。

2、思考时钟中断为什么使用 MMIO 计时器。

回答：时钟中断使用 MMIO 定时器，主要是因为定时器是一个外部设备，而不是 CPU 内部的一部分。CPU 只需要通过普通的读写指令访问 `mtime` 和 `mtimecmp`，就能完成定时器的配置和重装载，不需要额外设计专门的指令。这样既简化了 CPU 的设计，也方便在不同系统中复用同一套定时器模块。在 RISC-V 中，当 `mtime` 的值达到 `mtimecmp` 时，定时器会产生时钟中断信号，CPU 再根据相关 CSR 的设置决定是否响应。本实验中，`mtime` 和 `mtimecmp` 分别映射在 `0x3800bff8` 和 `0x38004000`，通过普通访存即可实现时钟中断的配置与触发。